

MINI PROJECT – 1

BOOKLOOP – SECOND HAND BOOK MARKETPLACE

A Digital Marketplace for Buying and Selling Second-Hand Books

Submitted in partial fulfillment of the requirements for

BACHELOR OF SCIENCE (B.Sc.) – COMPUTER SCIENCE

Semester V



THEEM COLLEGE OF ARTS, COMMERCE, AND SCIENCE

UNNIVERSITY OF MUMBAI

Submitted By

SAQUIB SHAIKH-

ROLL NO – 36

Under the Guidance of

SUJAL

ACADEMIC YEAR: 2026–2027

Github link <https://github.com/Shaikhsaquib7276/Bookloop>

Live url <https://bookloop.onrender.com/>

CERTIFICATE

This is to certify that the project entitled “BookLoop – A Second-Hand Book Marketplace Web Application” has been developed and documented as a Mini Project-I in accordance with the prescribed course requirements for Real-World Application Development.

Project Guide: _____ Signature: _____

Head of Department: _____ Signature: _____

Date: _____ Place: _____

DECLARATION

I/We hereby declare that the BookLoop project and this report have been prepared for academic submission. The project demonstrates the application of software engineering concepts including problem identification, requirements engineering, UML modeling, system architecture, implementation, testing, deployment and documentation.

Student Signature: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, faculty members and institution for their guidance and support during the development of BookLoop. We also acknowledge the open-source technologies and services used to implement the application, including Node.js, Express.js, EJS, MongoDB, Mongoose, Passport, Cloudinary, Git and GitHub.

ABSTRACT

BookLoop is a web-based second-hand book marketplace designed primarily for students. It provides a structured alternative to informal channels such as messaging groups, personal contacts and notice boards. Sellers can create book listings with title, author, description, price, condition, category and a cover image. Buyers can browse the catalogue, search and filter books, sort and paginate results, view book details and seller information, save books to a wishlist and submit reservation requests. Sellers can manage their listings and accept or reject reservation requests.

The application is implemented using Node.js and Express.js on the server side, EJS for server-rendered pages, MongoDB with Mongoose for persistent data, Passport Local for authentication and Cloudinary for book-cover storage. The project follows a modular architecture in which routes receive requests, middleware performs authentication/validation/ownership checks, controllers implement application logic, models communicate with MongoDB and EJS views render the result.

The report covers the complete Mini Project-I lifecycle: problem identification and feasibility, stakeholder analysis, functional and non-functional requirements, SDLC planning, WBS, Gantt chart, resource planning, the complete UML set, architecture design, implementation, testing, deployment, security/performance validation, documentation and future enhancements.

Keywords

BookLoop, second-hand books, student marketplace, Node.js, Express.js, EJS, MongoDB, Mongoose, Passport, Cloudinary, reservation system, web application.

TABLE OF CONTENTS

1. 1. Project Introduction and Overview
2. 2. Problem Identification and Feasibility Study
3. 3. Stakeholders, Scope and Objectives
4. 4. Requirement Engineering and SRS
5. 5. SDLC Planning and Project Management
6. 6. UML System Modeling
7. 7. System Architecture Design
8. 8. Database and Data Design
9. 9. Application Implementation
10. 10. Detailed User Workflows
11. 11. Testing Strategy and Test Cases
12. 12. Performance and Security Testing
13. 13. Deployment and Version Control
14. 14. Technical Documentation
15. 15. User Manual
16. 16. Screenshots / UI Evidence
17. 17. Course Outcomes Mapping
18. 18. Module Deliverables Checklist
19. 19. Limitations and Future Enhancements
20. 20. Conclusion
21. 21. References

1. PROJECT INTRODUCTION AND OVERVIEW

1.1 What is BookLoop?

BookLoop is a student-focused marketplace where students can publish and discover second-hand books. The core idea is simple: a student who owns a book they no longer need can list it, and another student who needs that book can discover the listing and request a reservation.

1.2 Why was BookLoop Developed?

Students often need textbooks for a semester or examination period but may not want to purchase expensive new copies. At the same time, other students may have usable books that are no longer required. Informal exchanges can work for small groups, but they become difficult to search, organize and track. BookLoop turns this informal exchange into a structured web application.

1.3 Current Feature Set

- Local signup, login and logout using Passport authentication.
- Create, edit, delete and view book listings.
- Book-cover image uploads using Cloudinary.
- Search, category/condition/price filtering, sorting and pagination.
- Seller profiles and listing status management.
- Personal dashboard with listing and wishlist statistics.
- Wishlist functionality for saving interesting books.
- Reservation requests where a buyer requests a listing and the seller accepts or rejects it.
- Responsive user interface for desktop and mobile layouts.
- Default BookLoop image when a seller does not upload a cover.

1.4 Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	JavaScript server runtime
Web Framework	Express.js	HTTP server, routing and middleware
View Layer	EJS + EJS-Mate	Server-side HTML rendering and layouts
Database	MongoDB	Persistent application data
ODM	Mongoose	Schemas, relationships and database queries
Authentication	Passport + Passport Local Mongoose	User authentication and sessions
Images	Cloudinary + Multer storage	Book-cover image upload/storage
Validation	Joi / express-validator	Input validation
Session / Messages	express-session + connect-flash	Login session and user feedback
Version Control	Git + GitHub	Source control and collaboration
UI	CSS + Bootstrap support + JavaScript	Responsive interface

2. PROBLEM IDENTIFICATION AND FEASIBILITY STUDY

2.1 Problem Statement

There is no centralized and structured mechanism in the target student environment for publishing, finding and reserving second-hand academic books. Students may have to search multiple chat groups or ask individuals, while sellers have no consistent listing format. The absence of search, filtering, seller information and reservation tracking makes the process inefficient.

2.2 Problem Justification

- Reduces the effort required to find affordable academic books.
- Creates a searchable catalogue instead of scattered messages.
- Allows sellers to provide standardized information about a book.
- Provides a clear buyer-to-seller reservation workflow.
- Encourages reuse of books and can reduce unnecessary expenditure.
- Provides an academic demonstration of a real-world marketplace system.

2.3 Scope Definition

In Scope	Out of Current Scope / Future
User accounts and sessions	Online payment processing
Book listing CRUD	Private buyer-seller messaging
Search/filter/sort/pagination	Server-side phone verification
Wishlist	Native mobile APK
Seller profiles	Advanced recommendation engine
Reservation requests	Automated delivery/logistics
Clouinary image storage	Full e-commerce order settlement

2.4 Feasibility Analysis

Feasibility	Assessment
Technical	High. Required technologies are mature, widely used and suitable for a student web project.
Economic	High. The application can be developed with open-source software and appropriate free/low-cost cloud tiers.
Operational	High. Students can use the system from a browser without installing special desktop software.
Schedule	Feasible for an iterative Mini Project because features can be developed in

	modules.
Legal / Ethical	The application should respect privacy, secure credentials and avoid storing unnecessary sensitive information.
BookLoop Mini Project – 9	

3. STAKEHOLDERS, SCOPE AND OBJECTIVES

3.1 Stakeholder Identification

Stakeholder	Needs / Responsibilities
Buyer / Student	Search books, view details, wishlist and request reservations.
Seller / Student	Create and manage listings, view requests and accept/reject them.
Project Team	Requirements, design, implementation, testing, deployment and documentation.
College / Institution	Academic evaluation and potential student benefit.
Database Service	Persistent storage for users, books and reservations.
Clouldinary	Storage and delivery of uploaded book-cover images.

3.2 Aim

To design and implement a practical student marketplace that simplifies the discovery and exchange of second-hand books while demonstrating a complete real-world software development lifecycle.

3.3 Objectives

- Identify and analyze a real institutional/social problem.
- Define functional and non-functional requirements.
- Model the system using UML and ER concepts.
- Build a modular Node.js/Express web application.
- Integrate MongoDB for persistent storage.
- Implement authentication, authorization and validation.
- Support book-cover image uploads.
- Implement search, filtering, wishlist and reservation workflows.
- Test the application using unit, black-box and integration approaches.
- Deploy and maintain the project using GitHub/version control.

3.4 Success Criteria

- A new user can register and log in.
- A seller can publish a valid book listing.
- A buyer can find a book using catalogue/search features.
- A buyer can request a reservation.
- A seller can accept/reject the request.
- Unauthorized users cannot perform protected owner actions.

- Uploaded images display correctly and a default image is shown when no image is uploaded.
- Core pages remain usable on mobile and desktop.

4. REQUIREMENT ENGINEERING AND SRS

4.1 Functional Requirements

ID	Requirement	Description
FR-01	User Registration	The system shall allow a new student to create an account.
FR-02	Authentication	The system shall authenticate users and maintain sessions.
FR-03	Listing Creation	An authenticated seller shall create a book listing.
FR-04	Listing Management	The owner shall edit or delete their listing.
FR-05	Catalogue	Students shall browse available book listings.
FR-06	Search	Students shall search using relevant listing text.
FR-07	Filtering	Students shall filter by category, condition and price range.
FR-08	Sorting/Pagination	The catalogue shall support sorting and paginated results.
FR-09	Book Details	A user shall view complete book and seller information.
FR-10	Wishlist	A logged-in user shall add/remove books from a wishlist.
FR-11	Dashboard	A user shall view relevant listing/wishlist information.
FR-12	Reservation	A buyer shall request a book reservation.
FR-13	Reservation Decision	A seller shall accept or reject a reservation request.
FR-14	Image Upload	A seller shall upload a book-cover image.
FR-15	Feedback	The system shall provide success/error feedback and error pages.

4.2 Non-Functional Requirements

NFR	Requirement
Usability	Navigation and forms should be understandable to students.

Responsiveness	Core pages should adapt to desktop and mobile screens.
Performance	Queries should be reasonably efficient and pagination should limit large result sets.
Security	Authentication, authorization, validation and protected ownership operations are required.
Reliability	Invalid requests should fail safely and show useful feedback.
Maintainability	Routes, controllers, models, middleware and views should remain modular.
Scalability	MongoDB and external image storage allow the project to grow beyond a local-only design.
Availability	Cloud hosting should be configured to keep the application accessible during normal use.

4.3 Use-Case Analysis

Use Case	Primary Actor	Precondition	Result
Register	Student	No account/session	Account created
Login	Student	Valid account	Authenticated session
Create Listing	Seller	Logged in	Book stored
Search Books	Student	Catalogue available	Matching results
View Book	Student	Book exists	Details displayed
Wishlist	Student	Logged in	Book saved/removed
Request Reservation	Buyer	Logged in; listing available	Pending request
Accept/Reject	Seller	Owns listing	Reservation status updated
Edit/Delete Listing	Seller	Logged in and owner	Listing changed/removed

4.4 Requirement Prioritization

Priority	Items
Must Have	Authentication, listing CRUD, database, catalogue, search/filter, validation, reservation.
Should Have	Wishlist, profiles, dashboard, image upload, responsive UI.
Could Have	Payment, messaging, phone verification,

	advanced recommendations.
Future / Optional	Mobile APK, logistics and advanced analytics.

4.5 Constraints and Assumptions

- Users are students or members of the intended marketplace community.
- Users need a browser and network connection.
- MongoDB and Cloudinary credentials are configured through environment variables.
- Current reservation is a request/approval workflow rather than an online payment transaction.
- Cloudinary is responsible for persistent book-cover storage.
- Production security should be strengthened further before handling real financial transactions.

5. SDLC PLANNING AND PROJECT MANAGEMENT

5.1 Selected SDLC Model

An Agile/iterative development model is appropriate for BookLoop. The project can be delivered incrementally: first the core catalogue and authentication, then listing management, then wishlist/reservations, followed by responsive UI, testing and deployment. This allows defects and usability issues to be corrected without redesigning the entire project.

5.2 Development Phases

Phase	Main Work
1. Problem Analysis	Identify problem, users, scope and feasibility.
2. Requirements	Prepare functional/non-functional requirements and SRS.
3. Design	Create UML, ER model, architecture and route/database design.
4. Implementation	Develop frontend, routes, controllers, models and integrations.
5. Testing	Unit, black-box, integration, validation, security and performance checks.
6. Deployment	Configure hosting, environment variables and GitHub workflow.
7. Documentation	Prepare technical report, user manual, screenshots and source documentation.

5.3 Work Breakdown Structure (WBS)

WBS	Work Package
1.0	Problem identification and feasibility
1.1	Stakeholders, scope and objectives
2.0	Requirement engineering and SRS
2.1	Functional/non-functional requirements
2.2	Use-case analysis and prioritization
3.0	UML and system design
3.1	Use Case, Class, Sequence, Activity, ER, Deployment
3.2	Architecture and database schema
4.0	Frontend implementation
5.0	Backend implementation
5.1	Authentication and authorization
5.2	Book listing/search/filter
5.3	Wishlist/profile/dashboard
5.4	Reservation workflow
6.0	Integration and testing
7.0	Deployment and GitHub version control

5.4 Gantt Chart

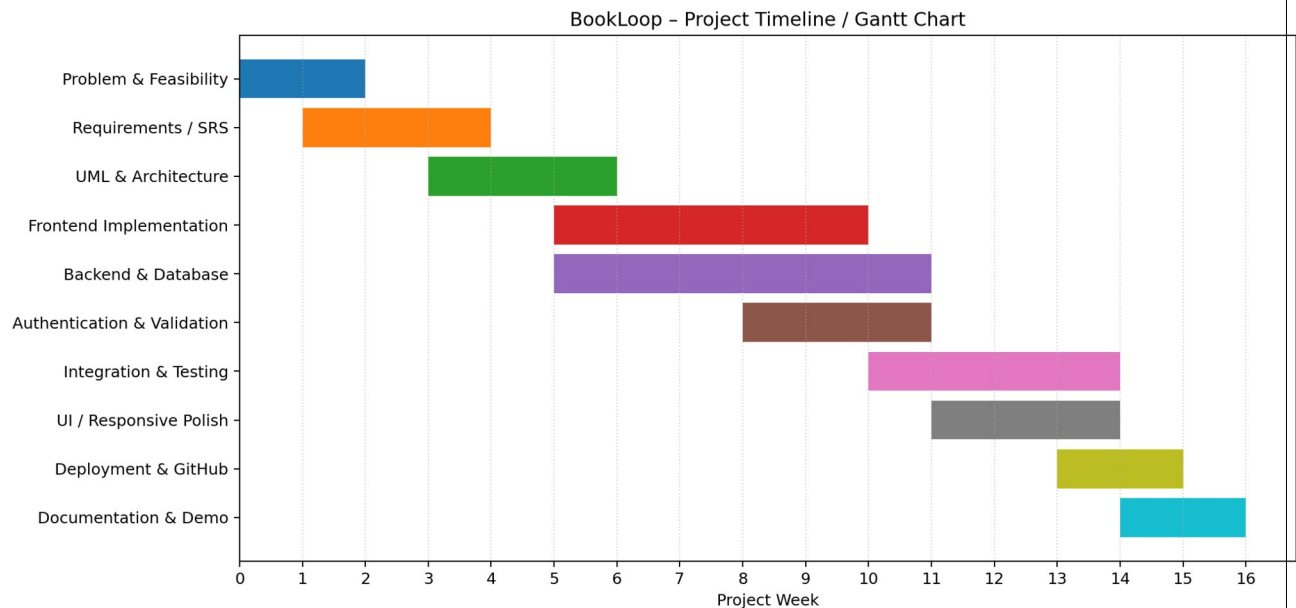


Figure 7: BookLoop Project Timeline / Gantt Chart

5.5 Resource Planning

Resource	Purpose
Student Developer(s)	Analysis, coding, testing and documentation.
Laptop/PC	Development and local testing.
Node.js	Server runtime.
VS Code / IDE	Source code development.
MongoDB / Atlas	Persistent database.
Cloudinary	Image storage.
Git + GitHub	Version control and repository.
Web Browser	Functional and UI testing.
Cloud Hosting	Production deployment.

6. UML SYSTEM MODELING

6.1 Use Case Diagram

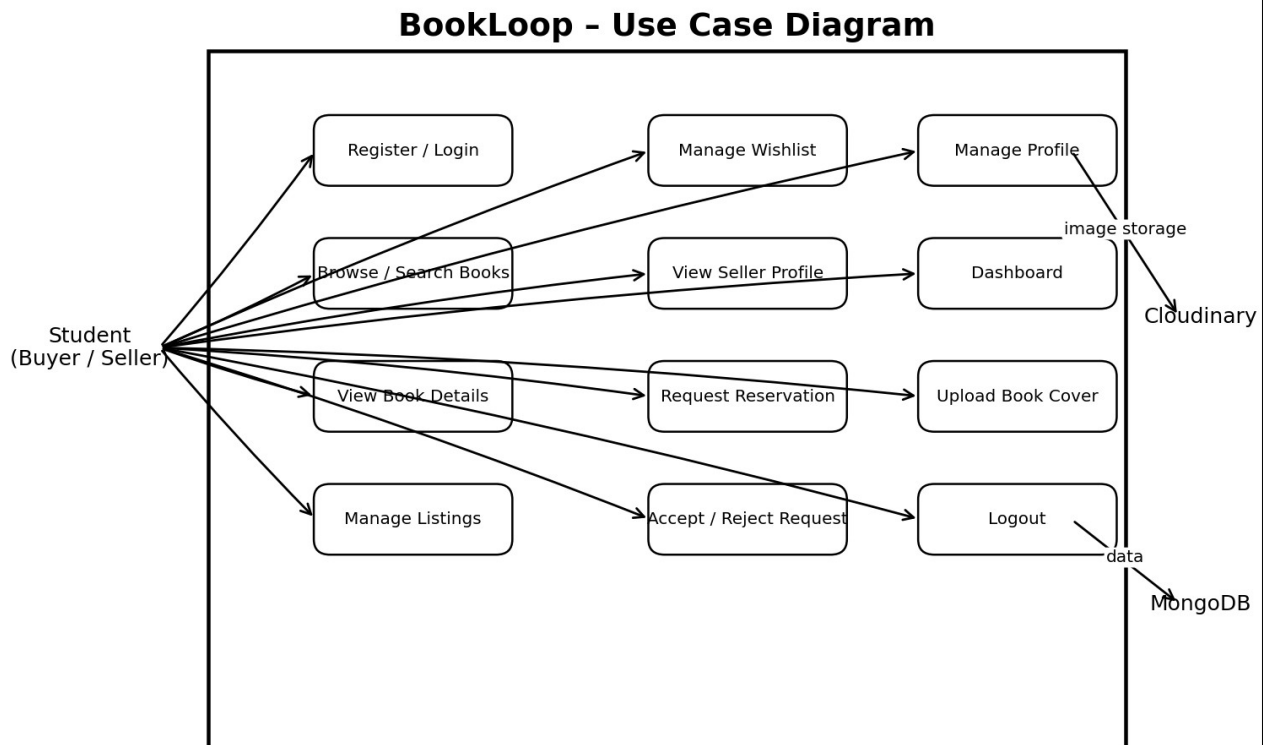


Figure 1: UML Use Case Diagram

The use case model identifies the Student as the main actor and shows the major interactions: registration/login, browsing, listing management, wishlist, profiles and reservation operations. Supporting services such as image storage and verification are shown where relevant.

6.2 Class Diagram

BookLoop - Class Diagram

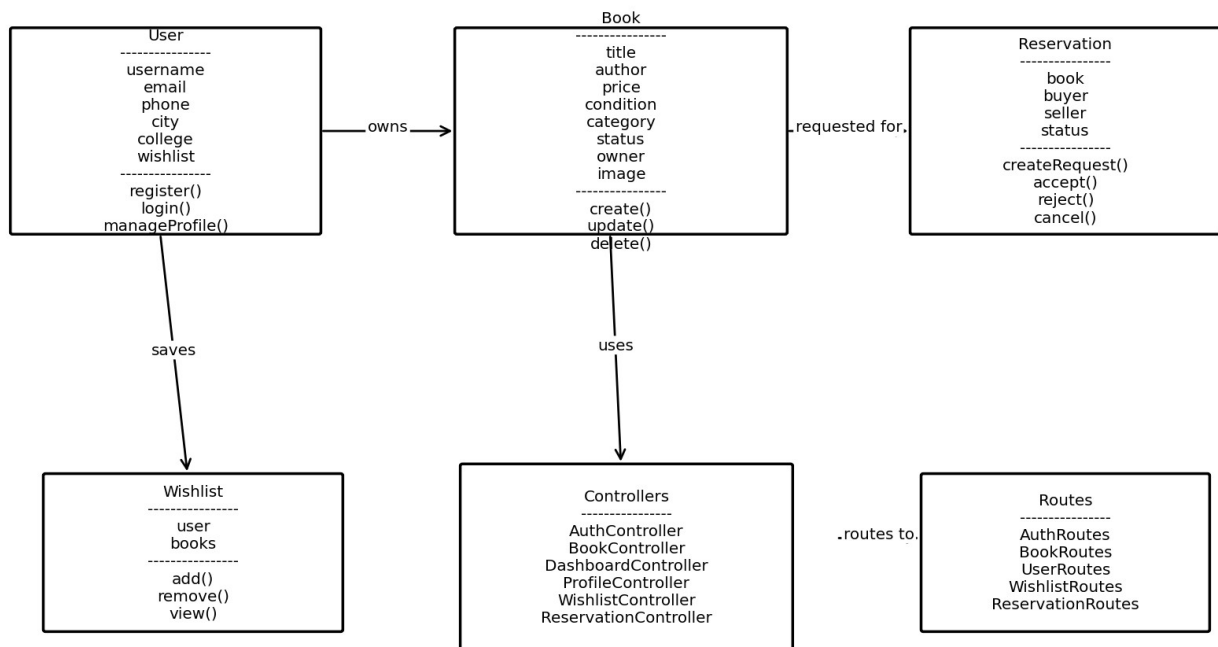


Figure 2: UML Class Diagram

The class model connects the main application concepts. User owns books, users can save books to a wishlist, and reservations connect a book with a buyer and seller. Routes, middleware and controllers are also represented as software components around the models.

6.3 Sequence Diagram – Reservation

BookLoop - Sequence Diagram: Reservation Request

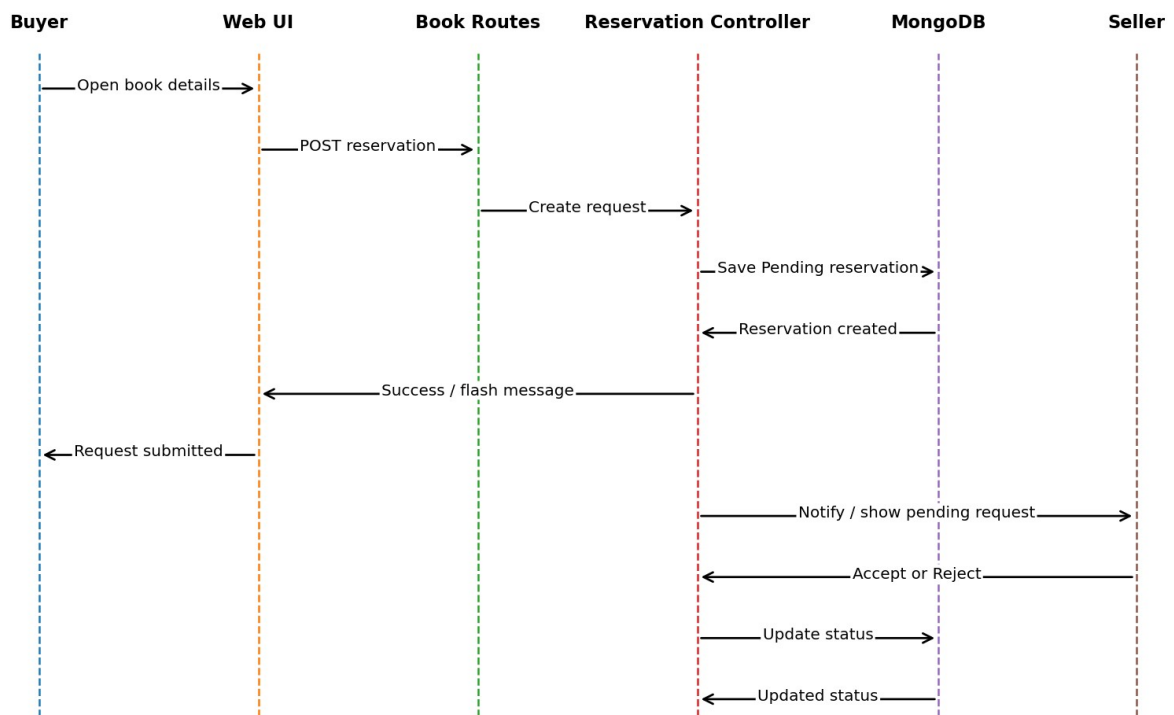


Figure 3: UML Sequence Diagram – Reservation Request

A buyer opens a book, submits a reservation request and the application validates the request before storing the reservation. The seller later processes the request. This diagram explains the time-ordered interaction among browser, routes/controllers, database and users.

6.4 Activity Diagram

BookLoop - Activity Diagram: Buy / Reserve Book

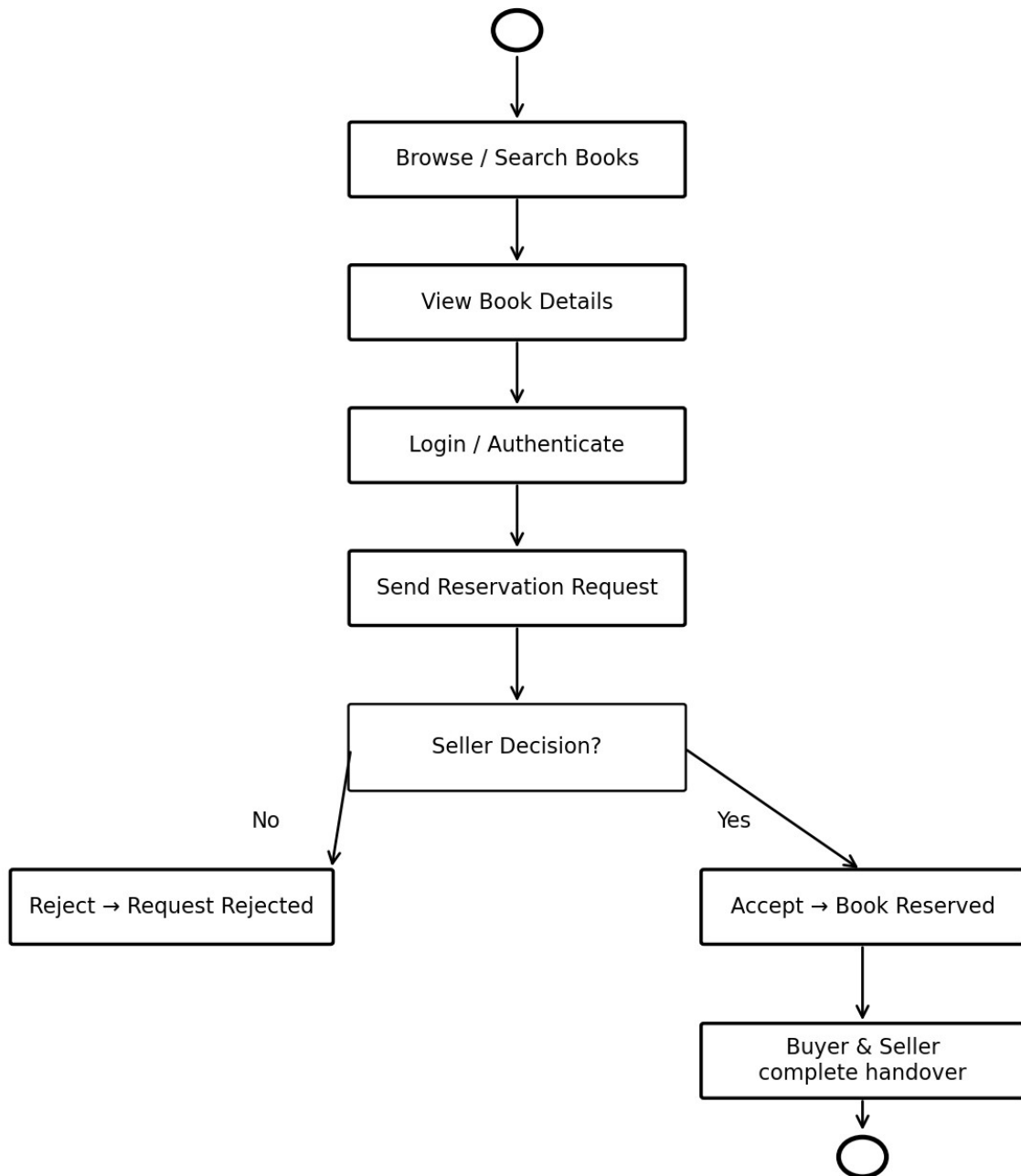


Figure 4: UML Activity Diagram – Buy/Reserve Book

The activity model starts with book discovery and continues through viewing details, authentication, reservation submission and the seller decision. It makes the business process understandable even to a reader who has not seen the source code.

6.5 ER Diagram

BookLoop - Entity Relationship (ER) Diagram

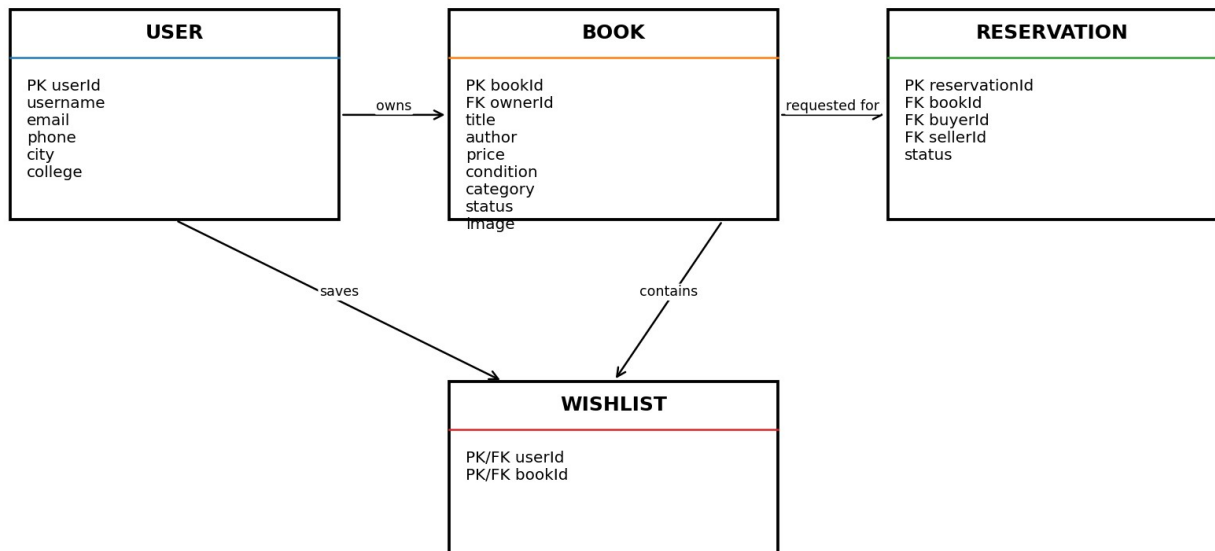


Figure 5: Entity Relationship Diagram

The ER model represents the persistent data: users, books, reservations and wishlist relationships. The User entity is related to owned books and reservation roles, while Book is connected to reservation and wishlist data.

6.6 Deployment Diagram

BookLoop - Deployment Diagram

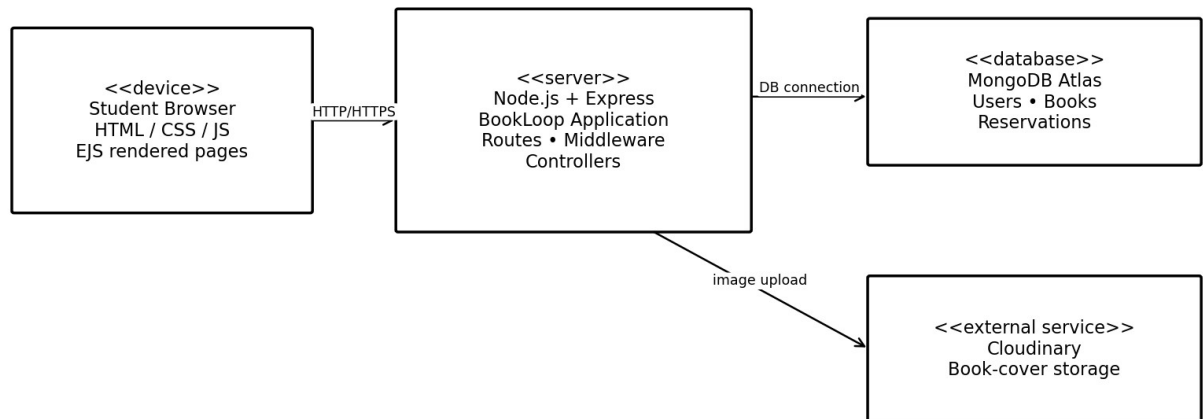


Figure 6: UML Deployment Diagram

The deployment diagram shows a student browser communicating with the Node.js/Express application. The application connects to MongoDB for persistent data and Cloudinary for book-cover images.

7. SYSTEM ARCHITECTURE DESIGN

7.1 High-Level Architecture

BookLoop follows a server-rendered MVC-style architecture. The browser sends HTTP requests to Express routes. Middleware performs cross-cutting tasks such as authentication, ownership checks and validation. Controllers implement application logic and call Mongoose models. Models communicate with MongoDB. EJS views generate the HTML response. Cloudinary is used when a book image is uploaded.

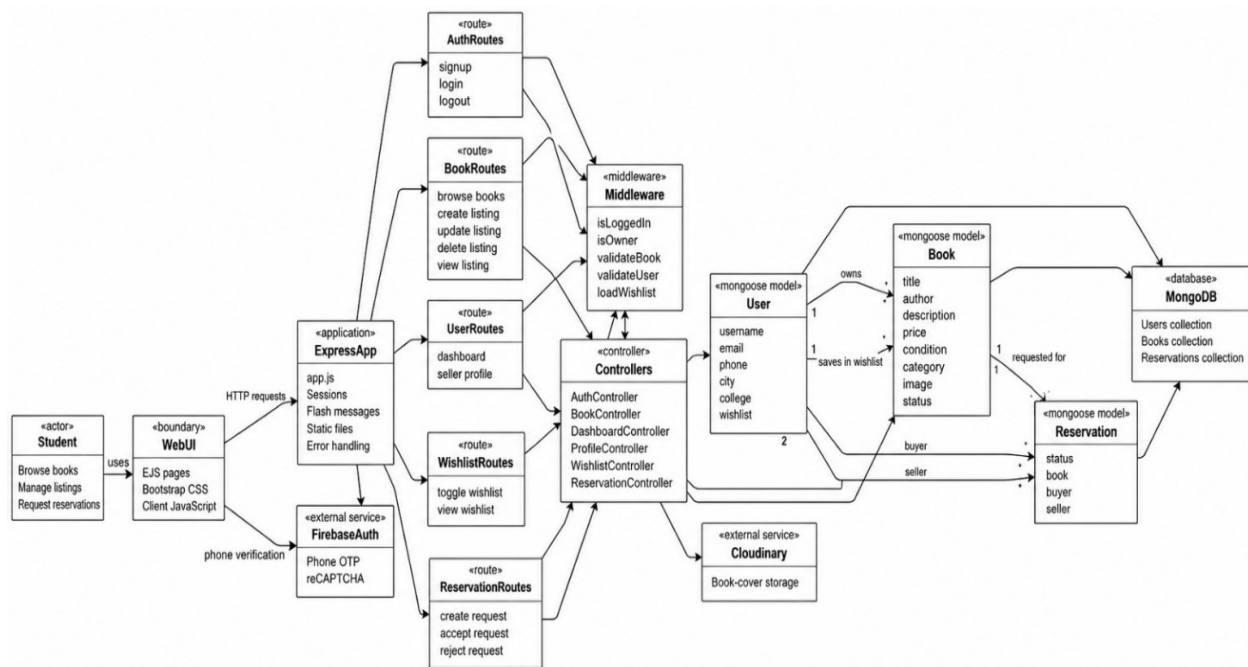


Figure 8: Supplied BookLoop Component/Software Architecture

7.2 Runtime Architecture

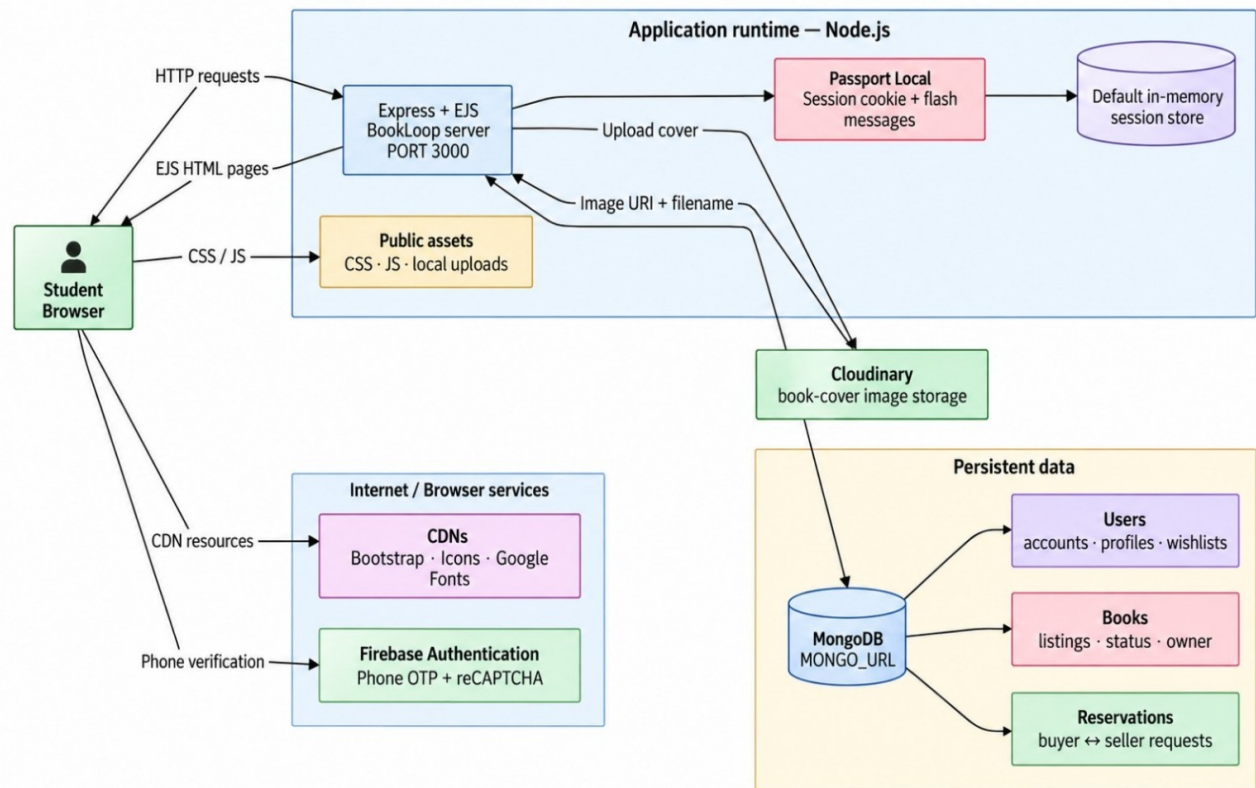


Figure 9: Supplied BookLoop Runtime Architecture

The runtime view shows the student browser, Node.js/Express/EJS application, Passport session handling, public assets, Cloudinary, MongoDB and external browser services. It provides a system-level view of where requests and data move during execution.

7.3 End-to-End Data Flow

BookLoop - End-to-End System Data Flow

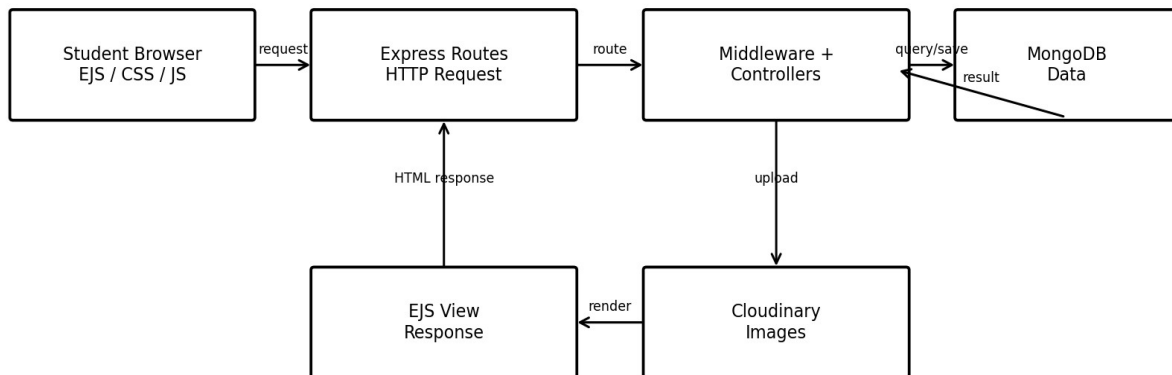


Figure 10: End-to-End Data Flow

The browser starts the interaction. Express routes select the correct workflow, middleware verifies access, controllers perform the operation, and models query or update MongoDB. For image uploads, the application communicates with Cloudinary and stores the returned image URI/filename with the book record. The controller then renders an EJS page for the browser.

7.4 Authentication and Authorization Flow

BookLoop - Authentication & Authorization Flow

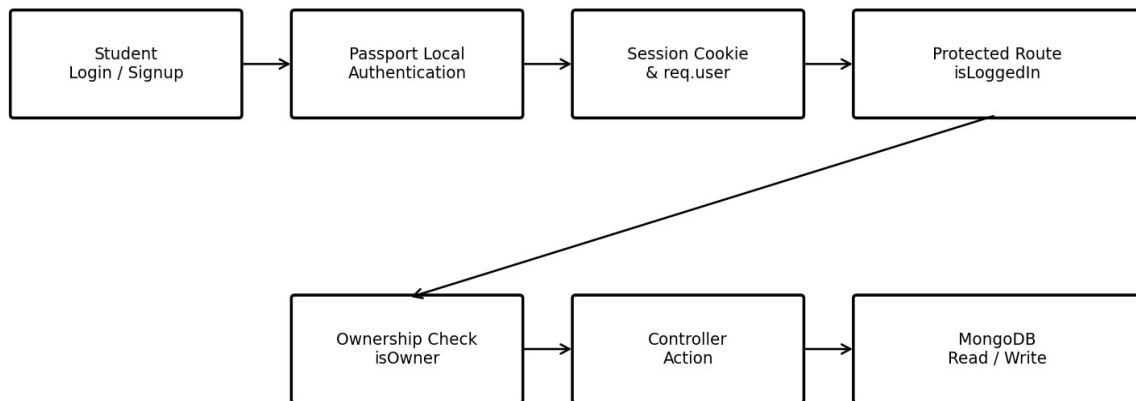


Figure 11: Authentication and Authorization Flow

Passport Local authenticates the user and a session identifies the authenticated request. Protected routes use login middleware. Owner-only operations additionally verify that the current user owns the requested book before allowing edits or deletion.

8. DATABASE AND DATA DESIGN

8.1 User Model

Field	Type / Meaning
username	Passport Local username / account identity
email	Unique required email
phone	User contact information
city	User location
college	College/institution
profileImage	Profile image reference
wishlist[]	References to Book documents
timestamps	Created/updated dates

8.2 Book Model

Field	Type / Meaning
title	Required string
author	Required string
description	Required string
price	Number, minimum 0
condition	New / Like New / Good / Fair / Poor
category	Book category
image.url	Cloudinary image URL or BookLoop default /book.jpg
image.filename	Uploaded file identifier
owner	Reference to User
status	Available / Reserved / Sold
timestamps	Created/updated dates

The current Book model explicitly uses /book.jpg as the default image and converts the known legacy external default image to the BookLoop default. This means a seller-uploaded Cloudinary image is preserved, while missing/legacy defaults render the BookLoop image. The application also exposes the root-level book.jpg through an explicit /book.jpg route. filecite

8.3 Reservation Model

Field	Meaning
book	Book being requested
buyer	User requesting the book
seller	Owner of the book
status	Pending / Accepted / Rejected /

	Cancelled
timestamps	Request creation/update dates

8.4 Relationships

- One User can own many Books.
- One User can have many wishlist references.
- One Book belongs to one owner User.
- One Book can be associated with reservation requests.
- A Reservation connects one Book, one buyer User and one seller User.
- MongoDB stores the collections while Mongoose manages references and schema rules.

8.5 Data Integrity

- Required fields prevent incomplete listings.
- Price has a non-negative constraint.
- Condition and status use controlled values.
- Ownership checks prevent another user from modifying a listing.
- Reservation status provides a controlled workflow.

9. APPLICATION IMPLEMENTATION

9.1 Project Structure

The repository is organized into application configuration, controllers, middleware, models, routes, views, public assets and supporting configuration. The major structure is:

Folder/File	Purpose
app.js	Main Express application, middleware, sessions, routes, error handling and server startup.
config/	Database, Passport and Cloudinary-related configuration.
controller/	Application/business logic for books, authentication, dashboard, profile, wishlist and reservations.
middleware/	Authentication, ownership, validation and wishlist loading.
models/	Mongoose schemas such as User, Book and Reservation.
routes/	HTTP route definitions for auth, books, dashboard, profile, wishlist and reservations.
views/	EJS pages, layouts and reusable view components.
public/	CSS, JavaScript and public static assets.
schemas.js	Validation schema definitions.
package.json	Dependencies and npm scripts.

9.2 Main Application Bootstrap

The application loads environment variables, creates an Express app, configures EJS/EJS-Mate, connects to the database, enables URL-encoded and JSON parsing, serves public assets, configures sessions, Passport and flash messages, then mounts authentication, book, dashboard, profile, wishlist and reservation routes. It also contains 404/error handling and starts the server on the configured PORT.

9.3 Dependencies

The current package configuration includes Express, EJS, EJS-Mate, Mongoose, MongoDB-related tooling, Passport, Passport Local, Passport Local Mongoose, Multer, Cloudinary storage, Joi, express-validator, sessions, flash messages, method-override, dotenv, bcrypt and Nodemon.

9.4 Route Modules

Route Module	Main Responsibility
auth.js	Signup, login and logout.
books.js	Browse, create, edit, delete and view books.
dashboard.js	Dashboard information.
profile.js	Seller/public profile and profile management.

wishlist.js	Toggle and view wishlist.
reservations.js	Create, accept and reject reservation requests.

The repository currently contains these six route modules under routes

9.5 Controller Layer

Controllers keep business logic separate from route definitions. Book controllers handle latest listings, search/filter/sort/pagination, book details, creation, editing and deletion. Other controllers handle authentication, dashboard, profiles, wishlist and reservations.

9.6 Middleware Layer

- isLoggedIn protects authenticated actions.
- isOwner protects seller-only listing operations.
- validateBook / validateUser enforce input rules.
- loadWishlist prepares wishlist information for the UI.
- Session and Passport middleware restore the authenticated user for each request.

9.7 Image Upload Implementation

A seller may upload a book cover. The book routes use Multer with Cloudinary-backed storage, and the controller stores the resulting URL/filename in the Book document. If no image is uploaded, the model's default image is used. This avoids the previous behavior where an external generic image could appear unexpectedly.

9.8 Default Image Logic

The Book model defines DEFAULT_BOOK_IMAGE as /book.jpg. Its getter returns that value when the stored image is missing or equals the known legacy external default URL. The schema enables getters in JSON/object conversion. The Express app explicitly serves the repository's root-level book.jpg at /book.jpg.

10. DETAILED USER WORKFLOWS

10.1 Seller Workflow – Listing a Book

22. Seller opens BookLoop and logs in.
23. Seller navigates to the new-listing form.
24. Seller enters title, author, description, price, condition and category.
25. Seller optionally selects a cover image.
26. Server validates the submitted fields.
27. If an image is provided, it is uploaded to Cloudinary.
28. The Book document is created with the authenticated user as owner.
29. The listing is displayed in the catalogue.
30. The seller can later edit or delete their own listing.

10.2 Buyer Workflow – Finding a Book

31. Buyer opens the book catalogue.
32. Buyer searches by keyword or applies filters.
33. Server builds the query and returns paginated results.
34. Buyer opens a book card to view details.
35. Buyer checks price, condition, description and seller information.
36. Buyer can add the book to the wishlist or request a reservation.

10.3 Reservation Workflow

37. Buyer must be authenticated.
38. Buyer opens an available book.
39. Buyer submits a reservation request.
40. Server validates that the request is allowed.
41. A Reservation record is stored with Pending status.
42. Seller sees the request in the reservation area.
43. Seller accepts or rejects the request.
44. The reservation status is updated accordingly.
45. Further payment/handover can happen outside the current system because online payment is not yet implemented.

10.4 What Happens When Someone Wants a Book?

The current application is not yet a complete online payment marketplace. It implements the discovery and reservation stage. Therefore, the practical flow is: Buyer → Find Book → View Details → Request/Reserve → Seller Reviews → Accept or Reject → Arrange handover/payment. Online payment and private messaging are planned enhancements.

10.5 Status Lifecycle

Status	Meaning
Available	Book can be requested.
Pending Reservation	A buyer has submitted a request.
Reserved	The seller has accepted/held the book.
Sold	The transaction has been completed

	outside/after the current reservation workflow.
Rejected / Cancelled	Reservation request is no longer active.

11. TESTING STRATEGY AND TEST CASES

11.1 Testing Objectives

- Verify that every major feature behaves as specified.
- Detect invalid inputs and unauthorized actions.
- Verify route-controller-database integration.
- Check responsive UI behavior.
- Verify image upload and default-image behavior.
- Confirm reservation state transitions.

11.2 Unit Testing

Unit testing focuses on isolated units such as validation logic, model rules, helper functions and controller behaviors. The repository exposes an npm test command using Node's test runner.

11.3 Black-Box Testing

Black-box testing checks the observable behavior of the application without requiring knowledge of its internal implementation. Examples include submitting a login form, searching for a book, creating a listing and requesting a reservation.

11.4 Integration Testing

Integration testing verifies that multiple modules work together: route → middleware → controller → model → MongoDB, and route/controller → Cloudinary for image uploads.

11.5 Test Case Matrix

ID	Scenario	Input	Expected Result	Result
TC-01	Valid registration	Enter valid account data	Account created	Pass/Record
TC-02	Invalid login	Wrong password	Login rejected	Pass/Record
TC-03	Create listing	Valid book details	Book stored/displayed	Pass/Record
TC-04	Invalid price	Negative price	Validation error	Pass/Record
TC-05	Upload image	Valid jpg/png/webp within limit	Cloudinary image displayed	Pass/Record
TC-06	No image	Create listing without cover	BookLoop default image displayed	Pass/Record
TC-07	Search	Enter book keyword	Matching books shown	Pass/Record
TC-08	Filter	Select condition/category/price	Filtered results shown	Pass/Record
TC-09	Wishlist	Toggle wishlist	Book	Pass/Record

			added/removed	
TC-10	Reservation	Authenticated buyer requests available book	Pending request created	Pass/Record
TC-11	Accept request	Seller accepts own request	Reservation updated	Pass/Record
TC-12	Reject request	Seller rejects request	Reservation updated	Pass/Record
TC-13	Unauthorized edit	Non-owner attempts edit	Access denied	Pass/Record
TC-14	Delete listing	Owner deletes	Listing removed	Pass/Record
TC-15	404	Open invalid route	Error page displayed	Pass/Record
TC-16	Mobile layout	Use narrow screen	Core UI remains usable	Pass/Record

11.6 Bug Tracking

Bug / Issue	Cause / Area	Resolution
Random/default external image	Legacy image URL in Book model	Getter now maps missing/legacy value to /book.jpg.
Root default image not directly public	book.jpg located at repository root	Explicit /book.jpg route added in app.js.
Responsive layout issues	Different viewport sizes	Responsive CSS and mobile filter improvements.
Inconsistent page styling	Separate page styles	Shared theme, navbar, cards and page-specific CSS.
Online payment not present	Not implemented in current scope	Documented as future enhancement rather than pretending it exists.

12. PERFORMANCE AND SECURITY TESTING

12.1 Basic Load Testing Plan

For an academic project, load testing can begin with repeated concurrent requests against high-use endpoints such as the home page, book catalogue and book detail pages. Observe response time, error rate and database behavior as the number of records and concurrent users increases.

Test Area	What to Measure	Expected Goal
Home page	Response time under repeated requests	Stable response without server errors
Catalogue	Search/filter query time	Acceptable response as data grows
Book details	Database lookup time	Fast lookup using indexed identifiers
Reservation	Concurrent requests	No inconsistent duplicate state
Image upload	Upload success/error rate	Valid files handled reliably

12.2 Input Validation

- Required book fields must be supplied.
- Price must be zero or greater.
- Condition/status values must belong to allowed enumerations.
- Image uploads are restricted by configured file type and size rules.
- Object IDs should be validated before database operations.
- Authenticated user identity must be used for ownership-sensitive operations.

12.3 Security Validation

- Passport-based authentication protects account access.
- Session cookies are configured as HTTP-only.
- Protected operations require authentication.
- Owner checks prevent users from editing/deleting another user's listing.
- Environment variables are used for secrets/configuration.
- Server-side validation is required even when client-side validation exists.
- Future production hardening should include CSRF protection, rate limiting, secure headers, stronger session storage and secure cookie settings appropriate to HTTPS.

12.4 Reliability and Error Handling

The Express application has a 404 handler and centralized error-rendering middleware. Flash messages communicate success/error states. This prevents users from receiving unhandled raw errors during normal invalid navigation or validation failures.

13. DEPLOYMENT AND VERSION CONTROL

13.1 Local Setup

46. Install Node.js and MongoDB or configure MongoDB Atlas.
47. Copy .env.example to .env.
48. Configure database, session and Cloudinary variables.
49. Install packages using npm install.
50. Run npm run dev during development.
51. Open the local application at http://localhost:3000.

13.2 Production Deployment

- Create the production service on the selected hosting platform.
- Connect the GitHub repository.
- Configure build/start commands and environment variables.
- Ensure MongoDB is reachable from the deployed service.
- Ensure Cloudinary credentials are present.
- Verify authentication, listing creation, image upload, search and reservation flows.
- Check the deployed logs for runtime errors.

13.3 GitHub Version Control

GitHub is a mandatory course requirement. The project repository is public and its default branch is main. Development work can be isolated on feature/fix branches, reviewed and then merged into main. The BookLoop UI improvements were developed on a separate ui-fix branch before being merged into main, preserving a controlled development workflow.

13.4 Current Repository Facts

Item	Value
Repository	https://github.com/Shaikhsaquib7276/Bookloop
Default branch	main
Application entry	app.js
Package scripts	start, dev, test
Main framework	Express.js
Database	MongoDB / Mongoose
Image storage	Cloudinary
Views	EJS / EJS-Mate

13.5 APK Build

The current BookLoop project is a browser-based web application, so an APK is not part of the current implementation. If the institution requires a mobile package in a later phase, the same backend could support a mobile client or a web-to-mobile wrapper.

14. TECHNICAL DOCUMENTATION

14.1 Request Lifecycle Example – Create Book

52. Browser submits the new-book form.
53. Express receives the HTTP POST request.
54. Authentication middleware checks the user session.
55. Upload middleware processes the optional image.
56. Validation middleware checks book data.
57. Book controller sets owner to the authenticated user.
58. If an image exists, the Cloudinary URL/filename is stored.
59. Mongoose saves the Book document in MongoDB.
60. Controller redirects/renderers the appropriate page and sends feedback.

14.2 Request Lifecycle Example – View Book

61. Browser requests /books/:id.
62. Book route receives the request.
63. Controller finds the Book by ID and populates the owner.
64. Related books and reservation information can be prepared.
65. Controller renders the EJS book detail view.
66. The view displays the stored image or BookLoop default image.

14.3 Request Lifecycle Example – Reservation

67. Authenticated buyer opens an available listing.
68. Reservation route receives the request.
69. Middleware confirms the buyer is logged in.
70. Controller checks the book and seller relationship.
71. A Reservation document is created/updated.
72. Seller later opens reservations.
73. Seller accepts or rejects the request.
74. Book/reservation status is reflected in the UI.

14.4 Important Configuration

Configuration	Purpose
MONGO_URL	MongoDB connection string.
SESSION_SECRET	Session signing secret.
Cloudinary credentials	Book-cover image upload/storage.
PORT	Server listening port; defaults to 3000 if not provided.

14.5 Source Code Quality Practices

- Separate route, controller, model and middleware responsibilities.
- Reusable EJS includes reduce repeated markup.
- Environment variables keep secrets out of source code.
- Git branches allow isolated changes.

- Validation and error handling reduce invalid states.
- Explicit image fallback prevents broken/undesired default images.

15. USER MANUAL

15.1 New User Registration

75. Open BookLoop.
76. Select Sign Up.
77. Enter the requested account information.
78. Submit the registration form.
79. After successful registration, log in with the account.

15.2 Login

80. Open Login.
81. Enter username/email and password as configured by the application.
82. Submit the form.
83. A successful login creates a session and exposes authenticated features.

15.3 Add a Book

84. Log in as a seller.
85. Open the Add/New Book page.
86. Enter book title and author.
87. Enter description, price, condition and category.
88. Select a cover image if available.
89. Submit the form.
90. Confirm the listing appears in the catalogue/dashboard.

15.4 Browse and Search

91. Open Browse Books.
92. Enter a search term if required.
93. Apply category, condition and price filters.
94. Choose a sorting option.
95. Move through pages if there are many results.
96. Open a book card to view complete details.

15.5 Wishlist

When logged in, a student can save an interesting book to the wishlist. The wishlist provides a personal collection of books the student may want to consider later.

15.6 Request a Book

97. Open the desired book's detail page.
98. Review price, condition, description and seller information.
99. Use the reservation/request action.
100. Wait for the seller to process the request.
101. Check reservation status from the appropriate reservations/dashboard area.

15.7 Seller Reservation Management

102. Open the reservations area while logged in as the seller.

- 103. Review the buyer and requested book.
- 104. Accept or reject the request.
- 105. The reservation status is updated accordingly.

15.8 Edit/Delete Your Listing

A seller can edit or delete a listing only when the ownership check confirms that the authenticated user owns the book. This protects other users' listings from unauthorized modification.

16. SCREENSHOTS / UI EVIDENCE

The following UI images are included as visual evidence of the BookLoop interface design. They show the dark-themed book detail/listing presentation and seller information components. For final college submission, additional screenshots of the actual deployed application can be added for login, signup, homepage, catalogue, dashboard, wishlist and reservations if required.

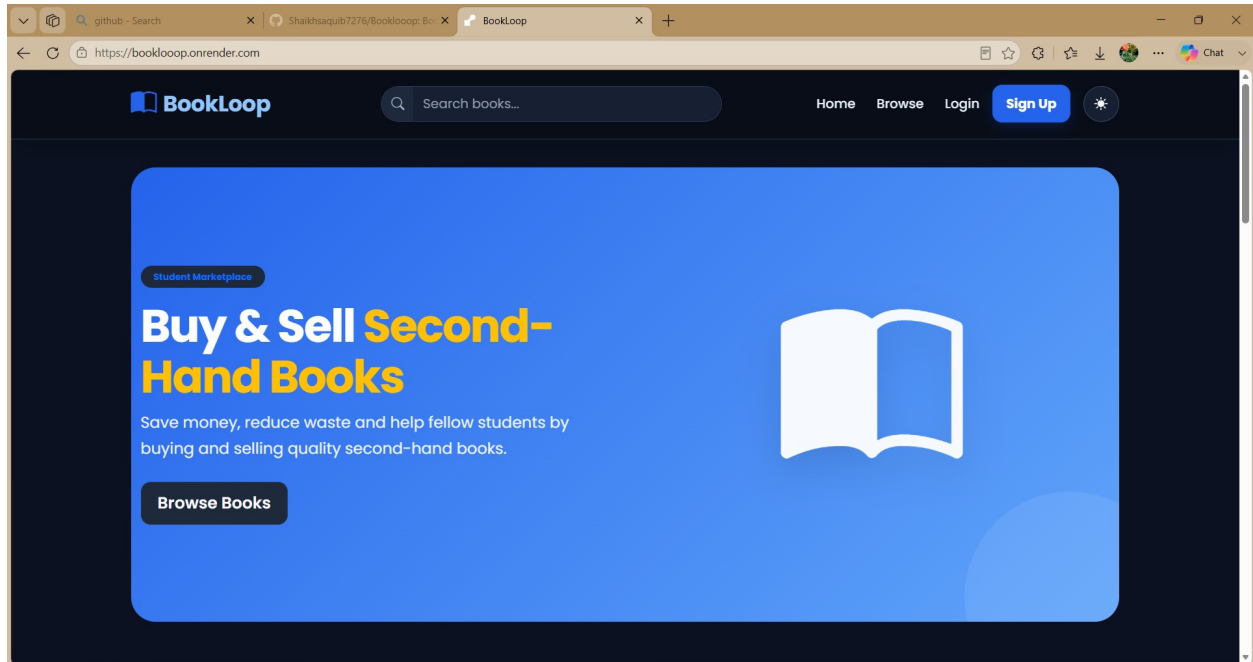


Figure 12: Book Detail UI – Example View 1

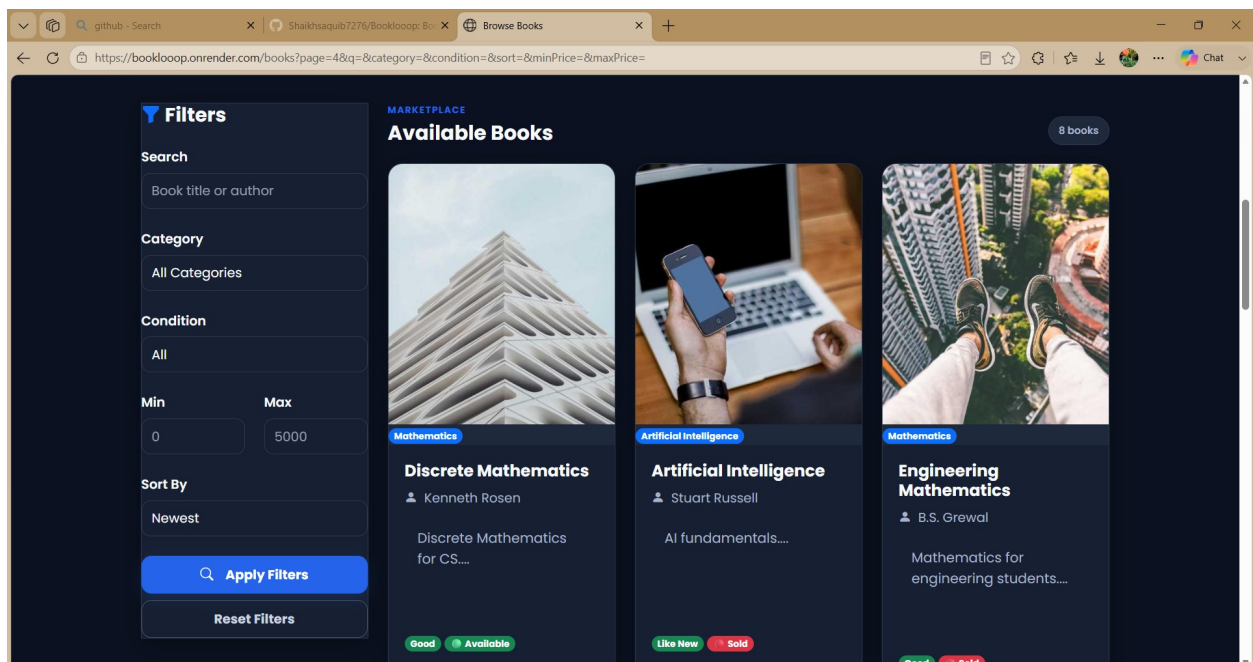
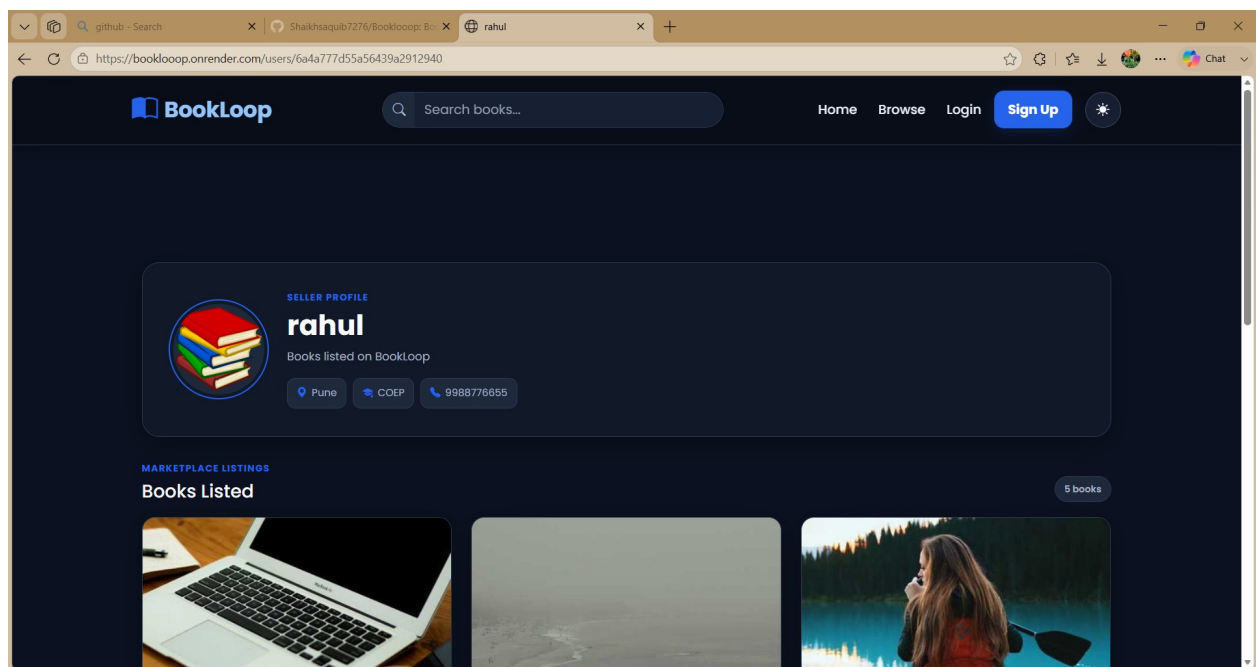
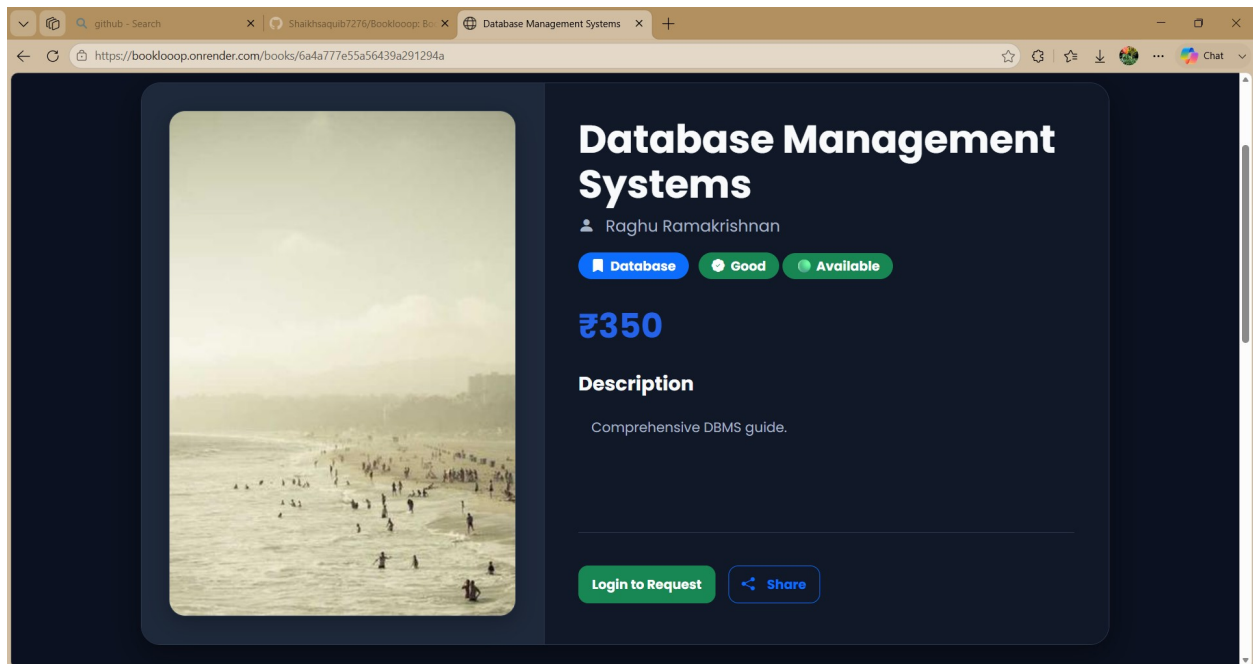


Figure 13: Book Detail UI – Example View 2



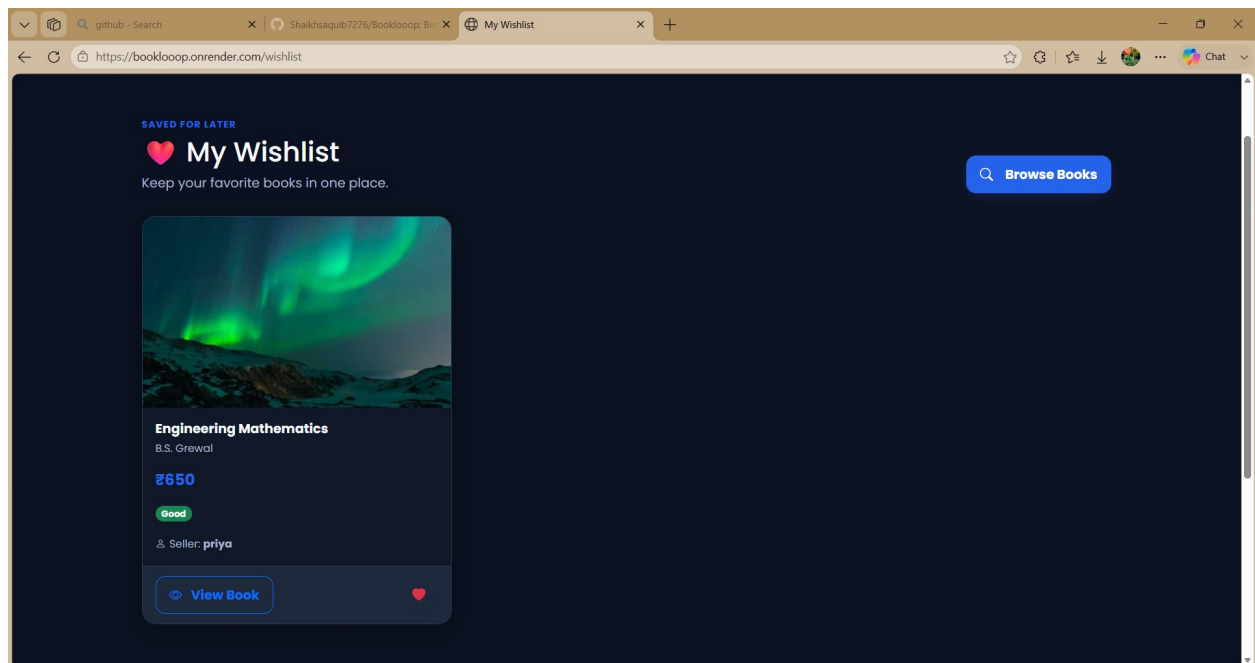
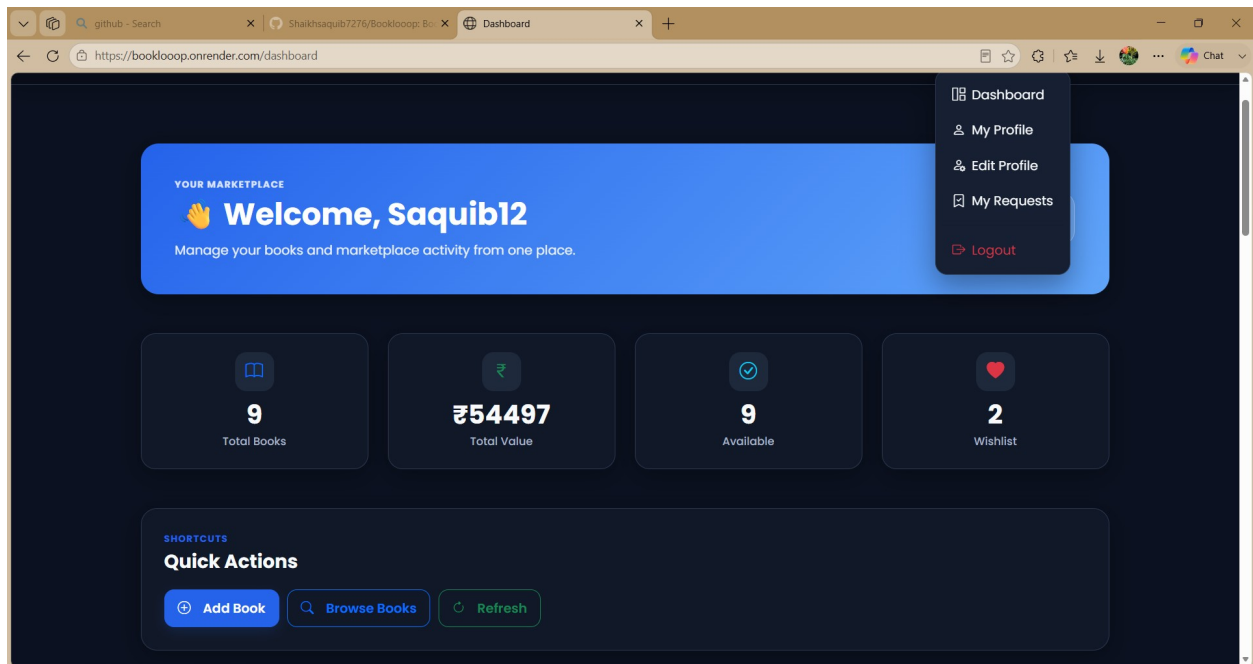


Figure 14: Book Detail UI – Example View 3

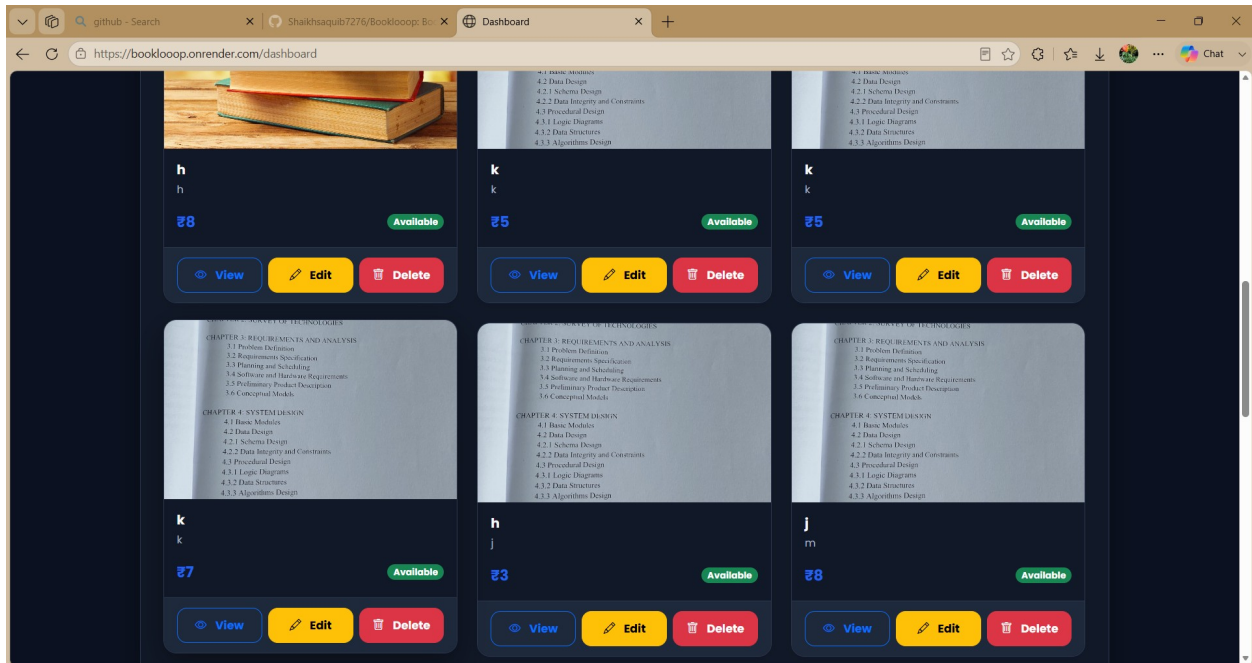


Figure 15: Book Detail UI – Example View 4

[Home](#)
[Browse](#)
[Sell Book](#)
[Wishlist](#)
[Saqulb12](#)

MARKETPLACE

Add a Book

Create a clean listing so other students can discover your book.

Title

Author

Description

Price (₹)

Condition

Category

Book Cover

No file chosen

Use a clear front-cover image. JPG, PNG and other common image formats are supported.

← Cancel

+ Add Book

Buy, sell and discover affordable second-hand books from your community.

Explore

- Home
- Browse Books
- Sell a Book
- Wishlist

Account

- Dashboard
- My Profile
- My Requests

© 2026 BookLoop

Built for students & book lovers



Java Programming Masterclass

Herbert Schildt

Programming

Like New

Sold

₹450

Description

Complete Java programming reference for beginners and professionals.

Login to Request

Share

17. COURSE OUTCOMES MAPPING

CO	Course Outcome Area	BookLoop Evidence
CO1	Identify and analyze real-world problems	Second-hand student book marketplace problem, stakeholders, feasibility and scope.
CO2	Develop skills in requirements, UML and architecture	SRS, functional/non-functional requirements, Use Case/Class/Sequence/Activity/ER/Deployment diagrams and architecture.
CO3	Implement software using modern tools	Node.js, Express, EJS, MongoDB, Mongoose, Passport, Cloudinary, validation and GitHub.
CO4	Integrate, test and evaluate	Unit/black-box/integration testing, test cases, bug tracking, performance and security validation.
CO5	Document and present project	Technical report, user manual, screenshots, source-code explanation, UML and deployment evidence.

17.1 Module 1 Compliance

Syllabus Point	Included in Report
Problem Identification & Feasibility	Yes – Section 2
Requirement Engineering	Yes – Section 4
SDLC Planning	Yes – Section 5
WBS	Yes – Section 5.3
Gantt Chart	Yes – Section 5.4
Resource Planning	Yes – Section 5.5
Use Case Diagram	Yes – Section 6.1
Class Diagram	Yes – Section 6.2
Sequence Diagram	Yes – Section 6.3
Activity Diagram	Yes – Section 6.4
ER Diagram	Yes – Section 6.5
Deployment Diagram	Yes – Section 6.6
System Architecture	Yes – Section 7
Database Schema	Yes – Section 8
API/Route Structure	Yes – Section 9.4

Security Considerations	Yes – Section 12
-------------------------	------------------

17.2 Module 2 Compliance

Syllabus Point	Included in Report
Frontend Implementation	Yes – Section 9
Backend Implementation	Yes – Section 9
Database Integration	Yes – Section 8/9
Authentication & Validation	Yes – Sections 9/12
Error Handling	Yes – Section 12
Unit Testing	Yes – Section 11
Black-box Testing	Yes – Section 11
Integration Testing	Yes – Section 11
Test Case Preparation	Yes – Section 11.5
Bug Tracking	Yes – Section 11.6
Cloud Deployment / Local Hosting	Yes – Section 13
APK Build	Discussed as not applicable to current web implementation
Server Configuration	Yes – Section 13
GitHub Version Control	Yes – Section 13
Basic Load Testing	Yes – Section 12.1
Input Validation Checks	Yes – Section 12.2
Security Validation	Yes – Section 12.3
Technical Report / User Manual / Screenshots / Source Documentation	Yes – Sections 14–16

19. LIMITATIONS AND FUTURE ENHANCEMENTS

19.1 Current Limitations

- Online payment is not implemented.
- Private buyer-to-seller messaging is not implemented.
- Phone verification is not fully implemented server-side.
- The current project is primarily a web application rather than a native mobile app.
- Advanced recommendation and analytics are not part of the current release.
- Production-scale security and load testing require further hardening.

19.2 Future Enhancements

- Integrate secure online payments.
- Add private buyer-seller chat.
- Implement verified phone numbers and OTP workflows.
- Add email/push notifications for reservation changes.
- Add advanced search and recommendation features.
- Introduce seller ratings and transaction feedback.
- Add admin moderation and reporting.
- Build a mobile application/APK using the same backend APIs.
- Add production-grade logging, monitoring, rate limiting and security headers.
- Add delivery/meeting-point coordination features.

20. CONCLUSION

BookLoop is a complete academic example of a real-world web application developed through the software engineering lifecycle. The project begins with a practical student problem and translates it into a defined scope, stakeholder model, requirements specification and feasibility study. It then uses UML and architecture diagrams to explain the system before implementation.

Technically, BookLoop demonstrates how a browser request travels through Express routes and middleware to controllers and Mongoose models, how MongoDB stores users/books/reservations, how Passport manages authentication, and how Cloudinary stores uploaded book-cover images. The reservation workflow provides a realistic marketplace interaction without incorrectly claiming that online payment is already implemented.

The project also addresses testing, error handling, security, responsive UI, deployment and GitHub version control. Therefore, a new developer or evaluator reading this report should be able to understand not only what BookLoop does, but also why it was built, how its modules interact, how its data is stored, how a user completes the main workflows and how the application can be extended in future.

21. REFERENCES

- Node.js documentation – server-side JavaScript runtime.
- Express.js documentation – web framework and middleware.
- MongoDB documentation – NoSQL database.
- Mongoose documentation – MongoDB object modeling.
- Passport.js documentation – authentication middleware.
- EJS documentation – server-side templating.
- Cloudinary documentation – media storage and delivery.
- Git documentation – distributed version control.
- GitHub documentation – repository and collaboration platform.
- BookLoop project source repository – Shaikhsaquib7276/Bookloop.

APPENDIX A – REPOSITORY IMPLEMENTATION NOTES

The current repository confirms the package scripts start, dev and test, with app.js as the main entry point.

The current app.js configures Express, EJS/EJS-Mate, sessions, Passport, flash messages, static files, the explicit /book.jpg default-image route, application routes and centralized error handling.

The current Book model contains the Book fields, owner/status relationships and the default-image getter logic described in this report.

The current route directory contains authentication, books, dashboard, profile, reservations and wishlist modules.